

Packet Capture and Protocol Analysis

Laboratory 3

Student Practical Laboratory Manual

Course	CORE102 TCP IP Networking and Linux Network Tools
Instructor	Aminu Idris AMCPN
Maximum score	100 marks
Gradebook category	Practical Laboratories
Estimated duration	5 to 6 hours
Platform	Kali Linux or Ubuntu Linux in an authorised laboratory environment
Version	September 2026

Student name _____Yohanna Gayus Kasa_____

Registration number _____ C11/26/OCO/17006_____

Submission date _____25th September, 2026_____

Purpose and Scenario

This laboratory builds practical packet-analysis skills with tcpdump, tshark and Wireshark. You will generate controlled ICMP, DNS, HTTP and SSH traffic, save packet captures and explain the protocol fields that establish communication.

A security analyst needs a repeatable capture of normal network activity before investigating anomalies. You must create a clean baseline, apply capture and display filters, reconstruct a short timeline and preserve evidence without collecting unrelated traffic.

Learning Outcomes

- Select an appropriate capture interface and document a capture plan.
- Use Berkeley Packet Filter expressions to limit tcpdump captures.
- Save and read PCAP files without altering their contents.
- Apply Wireshark display filters for hosts, ports, protocols and TCP flags.
- Identify ARP, ICMP, DNS, TCP and HTTP behaviour in packet evidence.
- Explain the TCP three-way handshake and recognise retransmission indicators.
- Create an evidence-based traffic timeline.

Environment and Prerequisites

- A Kali Linux VM with Wireshark, tcpdump, curl, dig and tshark.
- A snapshot and at least 2 GB of free disk space.
- Optional second Linux VM for the SSH exercise.
- Console access and sudo permission.
- A dedicated working directory named ~/core102_lab3.

Safety and Authorisation

- Capture only traffic generated by your own virtual machines inside the authorised laboratory.
- Do not enter real credentials into unencrypted test protocols.
- Use capture filters to reduce unrelated data and stop every capture as soon as the required packets are collected.
- Do not upload PCAP files containing third-party, personal or production traffic.
- Treat captured traffic as assessment evidence and preserve the original files.

Evidence Requirements

Item	Required evidence	Included
1	Capture plan and interface-selection evidence.	[]
2	Separate ICMP, DNS and local HTTP PCAP files.	[]
3	Optional authorised SSH PCAP or supplied instructor capture analysis.	[]
4	Wireshark screenshots showing required filters and packet details.	[]

Item	Required evidence	Included
5	Protocol observation tables and TCP handshake analysis.	[]
6	A final traffic timeline with supporting frame numbers.	[]

Part 1 Capture Plan and Baseline

1. Create the working directory and record the current time, host name, interfaces and routes.

```
(kali@kali)-[~/core102_lab3]
$ date
Thu Sep 24 05:53:16 AM EDT 2026
hostname
kali
whoami
kali
ip -br addr
ip route show default
default via 10.0.2.2 dev eth0 proto dhcp src 10.0.2.15 metric 100
sudo tcpdump -D
[sudo] password for kali:
1.eth0 [Up, Running, Connected]
2.eth1 [Up, Running, Connected]
3.any (Pseudo-device that captures on all interfaces) [Up, Running]
4.lo [Up, Running, Loopback]
5.bluetooth-monitor (Bluetooth Linux Monitor) [Wireless]
6.nflog (Linux netfilter log (NFLOG) interface) [none]
7.nfqueue (Linux netfilter queue (NFQUEUE) interface) [none]
8.dbus-system (D-Bus system bus) [none]
9.dbus-session (D-Bus session bus) [none]
(kali@kali)-[~/core102_lab3]
$
```

2. List the interfaces available to tcpdump.

```

(kali@kali)-[~/core102_lab3]
$ ip route show default
default via 10.0.2.2 dev eth0 proto dhcp src 10.0.2.15 metric 100

(kali@kali)-[~/core102_lab3]
$ sudo tcpdump -D
1.eth0 [Up, Running, Connected]
2.eth1 [Up, Running, Connected]
3.any (Pseudo-device that captures on all interfaces) [Up, Running]
4.lo [Up, Running, Loopback]
5.bluetooth-monitor (Bluetooth Linux Monitor) [Wireless]
6.nflog (Linux netfilter log (NFLOG) interface) [none]
7.nfqueue (Linux netfilter queue (NFQUEUE) interface) [none]
8.dbus-system (D-Bus system bus) [none]
9.dbus-session (D-Bus session bus) [none]

(kali@kali)-[~/core102_lab3]
$ █

```

- Choose the active network interface for ICMP and DNS and the loopback interface for the local HTTP capture.

Traffic type	Interface chosen	Reason
Traffic Type	Interface chosen	Reason
ICMP (ping to gateway)	eth0	Real traffic between VM and gateway crosses the Ethernet interface
DNS (dig queries)	eth0	DNS queries leave the VM through the default route on eth0
Local HTTP (Python server)	lo	Server and client both live on the same VM → loopback traffic only, no network required
SSH (if 2nd VM available)	private iface (e.g. eth1)	SSH session between two VMs crosses the private network, not the NAT eth0

- Record the capture purpose, interface, filter, expected traffic and stop condition before starting each capture.

```

mkdir -p ~/core102_lab3 && cd ~/core102_lab3
date
hostname
ip -br addr
ip route
sudo tcpdump -D
df -h .

```

Capture	Interface	Capture filter	Expected packets	Stop condition
ICMP	eth0	icmp	Echo request and reply	10 packets
DNS	eth0	udp port 53 or tcp port 53	Query and response	20 packets

Capture	Interface	Capture filter	Expected packets	Stop condition
HTTP	lo	tcp port 8080	Handshake and HTTP request	After one request
SSH	private iface	tcp port 22	Handshake and encrypted session	After login and exit

Part 2 ICMP Capture

1. Store the active interface name in NET_IFACE and the default gateway in GATEWAY.

```
(kali@kali)-[~/core102_lab3]
$ cd ~/core102_lab3
sudo tcpdump -nn -tttt -r core102_icmp.pcap
[sudo] password for kali:
reading from file core102_icmp.pcap, link-type EN10MB (Ethernet), snapshot length 262144
2026-09-24 06:15:38.991082 IP 10.0.2.15 > 10.0.2.2: ICMP echo request, id 10933, seq 1, length 64
2026-09-24 06:15:38.991199 IP 10.0.2.2 > 10.0.2.15: ICMP echo reply, id 10933, seq 1, length 64
2026-09-24 06:15:40.008999 IP 10.0.2.15 > 10.0.2.2: ICMP echo request, id 10933, seq 2, length 64
2026-09-24 06:15:40.009455 IP 10.0.2.2 > 10.0.2.15: ICMP echo reply, id 10933, seq 2, length 64
2026-09-24 06:15:41.032536 IP 10.0.2.15 > 10.0.2.2: ICMP echo request, id 10933, seq 3, length 64
2026-09-24 06:15:41.033640 IP 10.0.2.2 > 10.0.2.15: ICMP echo reply, id 10933, seq 3, length 64
2026-09-24 06:15:42.033893 IP 10.0.2.15 > 10.0.2.2: ICMP echo request, id 10933, seq 4, length 64
2026-09-24 06:15:42.034276 IP 10.0.2.2 > 10.0.2.15: ICMP echo reply, id 10933, seq 4, length 64
2026-09-24 06:18:22.011455 IP 10.0.2.15 > 10.0.2.2: ICMP echo request, id 10934, seq 1, length 64
2026-09-24 06:18:22.011811 IP 10.0.2.2 > 10.0.2.15: ICMP echo reply, id 10934, seq 1, length 64
```

2. Start a ten-packet ICMP capture and save it to core102_icmp.pcap.

```
(kali@kali)-[~/core102_lab3]
$ tcpdump -ni eth0 -c 10 -s0 -w core102_icmp.pcap icmp
[1] ++ done maxm wireshark core102_icmp.pcap
(kali@kali)-[~/core102_lab3]
$ cd ~/core102_lab3
sudo tcpdump -ni eth0 -c 10 -s0 -w core102_icmp.pcap icmp
[sudo] password for kali:
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C0 packets captured
0 packets received by filter
0 packets dropped by kernel
```

3. In a second terminal, ping the gateway four times.

```
(kali@kali)-[~]core102_lab3
$ ping -c 4 10.0.2.2
PING 10.0.2.2 (10.0.2.2) 56(84) bytes of data:
64 bytes from 10.0.2.2: icmp_seq=1 ttl=255 time=0.408 ms
64 bytes from 10.0.2.2: icmp_seq=2 ttl=255 time=0.998 ms
64 bytes from 10.0.2.2: icmp_seq=3 ttl=255 time=0.975 ms
64 bytes from 10.0.2.2: icmp_seq=4 ttl=255 time=0.332 ms

— 10.0.2.2 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3025ms
rtt min/avg/max/mdev = 0.332/0.678/0.998/0.309 ms

(kali@kali)-[~]
$
```

4. Read the saved capture with numeric addresses and identify each request/reply pair.

```
NET_IFACE=$(ip route show default | awk 'NR==1 {print $5}')
GATEWAY=$(ip route show default | awk 'NR==1 {print $3}')
echo "Interface: $NET_IFACE"
echo "Gateway: $GATEWAY"
sudo tcpdump -ni "$NET_IFACE" -c 10 -s0 -w core102_icmp.pcap icmp

# Second terminal
ping -c 4 "$GATEWAY"

# Read after capture completes
sudo tcpdump -nn -tttt -r core102_icmp.pcap
```

The image shows the Wireshark network protocol analyzer interface. The main pane displays a list of 10 captured packets, all of which are ICMP Echo (ping) requests and replies. The packets alternate between source IP 10.0.2.15 and destination IP 10.0.2.2. The first packet (No. 1) is an Echo request from 10.0.2.15 to 10.0.2.2. The second packet (No. 2) is an Echo reply from 10.0.2.2 to 10.0.2.15. This pattern continues for the remaining packets. The packet details pane on the right shows the structure of the first packet: Ethernet II, Internet Protocol Version 4, and Internet Control Message Protocol. The packet bytes pane shows the raw hex and ASCII data.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.2.15	10.0.2.2	ICMP	98	Echo (ping) request
2	0.000117	10.0.2.2	10.0.2.15	ICMP	98	Echo (ping) reply
3	1.017917	10.0.2.15	10.0.2.2	ICMP	98	Echo (ping) request
4	1.018373	10.0.2.2	10.0.2.15	ICMP	98	Echo (ping) reply
5	2.041454	10.0.2.15	10.0.2.2	ICMP	98	Echo (ping) request
6	2.042558	10.0.2.2	10.0.2.15	ICMP	98	Echo (ping) reply
7	3.042811	10.0.2.15	10.0.2.2	ICMP	98	Echo (ping) request
8	3.043194	10.0.2.2	10.0.2.15	ICMP	98	Echo (ping) reply
9	163.020373	10.0.2.15	10.0.2.2	ICMP	98	Echo (ping) request
10	163.020729	10.0.2.2	10.0.2.15	ICMP	98	Echo (ping) reply

Frame	Source	Destination	ICMP type	Identifier or sequence	Meaning
1	10.0.2.15	10.0.2.2	8 (echo request)	id=____ seq=1	Client → gateway ping no.1
2	10.0.2.2	10.0.2.15	0 (echo reply)	id=____ seq=1	Gateway replies to no.1
3	10.0.2.15	10.0.2.2	8	id=____ seq=2	Client → gateway ping no.2
4	10.0.2.2	10.0.2.15	0	id=____ seq=2	Gateway replies to no.2

Part 3 DNS Capture

1. Start a DNS capture on the active interface.

```
(kali@kali)~/core102_lab3
$ sudo tcpdump -nn -vv -r core102_dns.pcap '(udp port 53 or tcp port 53)'
reading from file core102_dns.pcap, link-type EN10MB (Ethernet), snapshot length 262144
07:34:48.325916 IP (tos 0x0, ttl 64, id 45884, offset 0, flags [none], proto UDP (17), length 80)
  10.0.2.15.60957 > 192.168.1.1.53: [bad udp cksum 0xce05 → 0x56ee!] 26640+ [1au] A? example.com. ar: . OPT UDPsize=1232 [COOKIE cbe3fe7024efc082] (52)
07:34:48.420855 IP (tos 0x0, ttl 64, id 14, offset 0, flags [none], proto UDP (17), length 100)
  192.168.1.1.53 > 10.0.2.15.60957: [udp sum ok] 26640 q: A? example.com. 2/0/1 example.com. A 104.20.23.154, example.com. A 172.66.147.243 ar: . OPT UDPsize=512 (72)
07:34:48.460239 IP (tos 0x0, ttl 64, id 57379, offset 0, flags [none], proto UDP (17), length 78)
  10.0.2.15.55661 > 192.168.1.1.53: [bad udp cksum 0xce03 → 0xf907!] 5812+ [1au] MX? gmail.com. ar: . OPT UDPsize=1232 [COOKIE 040683ef9f5bbbd3] (50)
07:34:48.515688 IP (tos 0x0, ttl 64, id 15, offset 0, flags [none], proto UDP (17), length 189)
  192.168.1.1.53 > 10.0.2.15.55661: [udp sum ok] 5812 q: MX? gmail.com. 5/0/1 gmail.com. MX alt1.gmail-smtp-in.l.google.com. 10, gmail.com. MX alt4.gmail-smtp-in.l.google.com. 40, gmail.com. MX gmail-smtp-in.l.google.com. 5, gmail.com. MX alt3.gmail-smtp-in.l.google.com. 30, gmail.com. MX alt2.gmail-smtp-in.l.google.com. 20 ar: . OPT UDPsize=512 (161)
(kali@kali)~/core102_lab3
```

2. Generate an A query and an MX query with dig.

```

Session  Actions  Edit  View  Help
Core102 -> core102 show defaults: core102 [initial 50]
(kali@kali)-[~]
$ dig example.com A
dig gmail.com MX

; <<>> DiG 9.20.27-2-Debian <<>> example.com A
;; global options: +cmd
;; Got answer:
;; -->HEADER<-- opcode: QUERY, status: NOERROR, id: 26640
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags::; udp: 512
;; QUESTION SECTION:
;example.com.                IN      A
;; ANSWER SECTION:
example.com. 192.168.1.1#53 IN      A      104.20.23.154
example.com. 192.168.1.1#53 IN      A      172.66.147.243

;; Query time: 95 msec
;; SERVER: 192.168.1.1#53(192.168.1.1) (UDP)
;; WHEN: Thu Sep 24 07:34:48 EDT 2026
;; MSG SIZE rcvd: 72

; <<>> DiG 9.20.27-2-Debian <<>> gmail.com MX
;; global options: +cmd
;; Got answer:
;; -->HEADER<-- opcode: QUERY, status: NOERROR, id: 5812
;; flags: qr rd ra; QUERY: 1, ANSWER: 5, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags::; udp: 512

```

3. Read the capture and identify the client source port, server port, transaction pair and response status.
4. If the system uses encrypted DNS and port 53 traffic is absent, document that finding and use the instructor-provided DNS capture or temporarily query an authorised DNS server directly.

```

sudo tcpdump -ni "$NET_IFACE" -c 20 -s0 -w core102_dns.pcap '(udp port 53 or tcp port 53)'

# Second terminal
dig example.com A
dig gmail.com MX

# Read after capture completes
sudo tcpdump -nn -vv -r core102_dns.pcap '(udp port 53 or tcp port 53)'

```

Query name	Type	Client port	Server	Response code	Answer summary
example.com	A	(e.g. 45123)	53	NOERROR (0)	A (IP address)
gmail.com	MX	(e.g. 51234)	53	NOERROR (0)	MX (mail server names)

Part 4 Local HTTP Capture

This exercise uses loopback traffic so it works without internet access. Use three terminals.

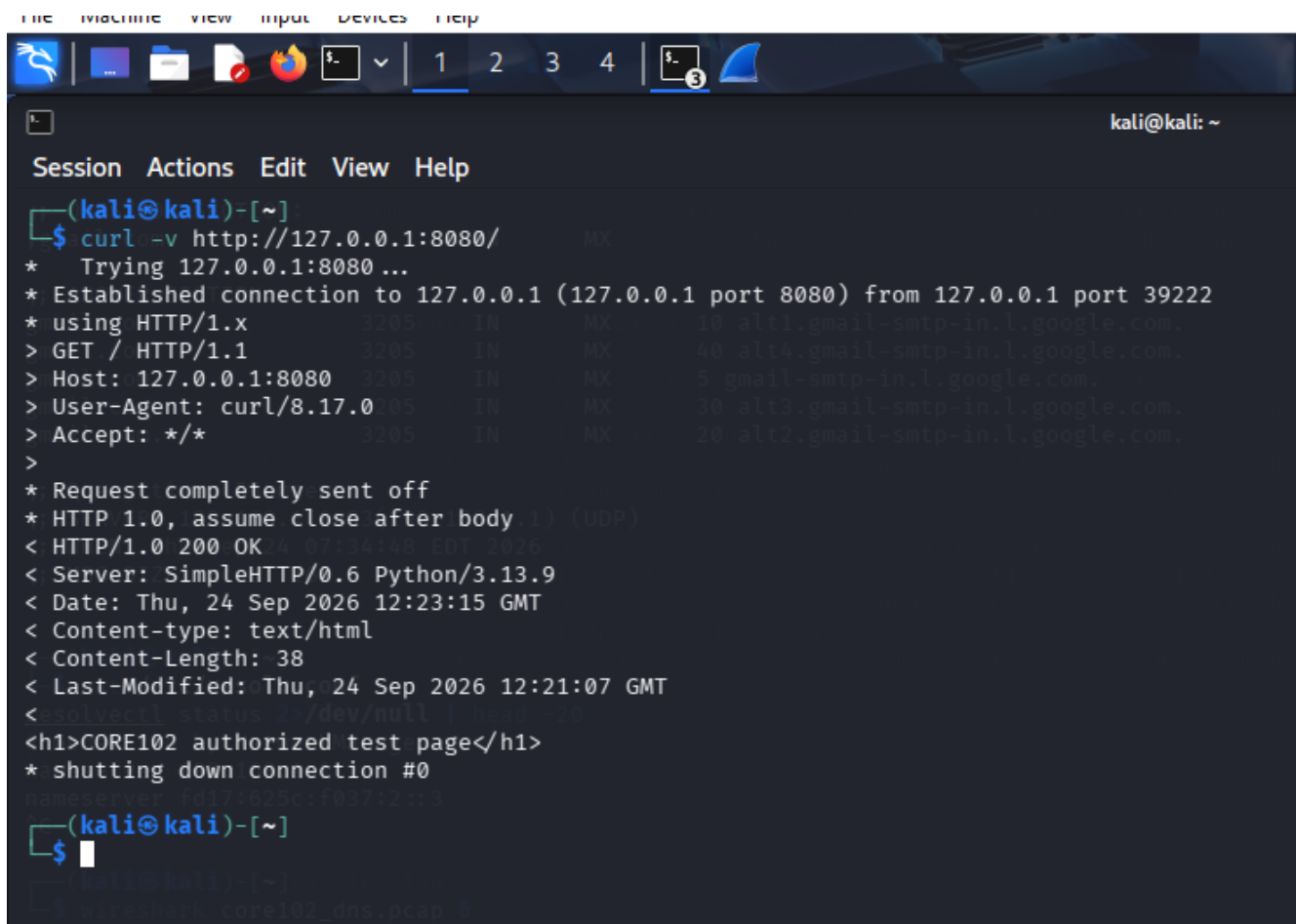
1. In Terminal 1, start a Python HTTP server bound only to 127.0.0.1 on port 8080.

```
(kali㉿kali)-[~/core102_lab3]
$ cd ~/core102_lab3
echo '<h1>CORE102 authorized test page</h1>' > index.html
python3 -m http.server 8080 --bind 127.0.0.1
Serving HTTP on 127.0.0.1 port 8080 (http://127.0.0.1:8080/) ...
127.0.0.1 - - [24/Sep/2026 08:23:15] "GET / HTTP/1.1" 200 ->shot length
```

2. In Terminal 2, capture loopback traffic to core102_http.pcap.

```
(kali㉿kali)-[~]
$ cd ~/core102_lab3
sudo tcpdump -ni lo -s0 -w core102_http.pcap 'tcp port 8080'
[sudo] password for kali:
tcpdump: listening on lo, link-type EN10MB (Ethernet), snapshot length 262144 bytes
```

3. In Terminal 3, request the server's index page with curl and then stop the capture.



```
File Machine View Input Devices Help
1 2 3 4 5
kali@kali: ~
Session Actions Edit View Help
(kali㉿kali)-[~]
$ curl -v http://127.0.0.1:8080/
* Trying 127.0.0.1:8080 ...
* Established connection to 127.0.0.1 (127.0.0.1 port 8080) from 127.0.0.1 port 39222
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 127.0.0.1:8080
> User-Agent: curl/8.17.0
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body 1) (00P)
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6 Python/3.13.9
< Date: Thu, 24 Sep 2026 12:23:15 GMT
< Content-type: text/html
< Content-Length: 38
< Last-Modified: Thu, 24 Sep 2026 12:21:07 GMT
< /dev/null
<h1>CORE102 authorized test page</h1>
* shutting down connection #0
(kali㉿kali)-[~]
$
```

4. Read the PCAP and identify the TCP handshake, HTTP request and connection closure.

```

kali@kali: ~/core102_lab3
Session Actions Edit View Help
.2.Ai..h
08:23:15.832749 IP 127.0.0.1.8080 > 127.0.0.1.39222: Flags [P.], seq 1:186, ack 79, win 256, options [nop,nop,TS val 2603746180 ecr 1776672616], length 185: HTTP: HTT
P/1.0 200 OK
E...^..@..@..k.....6Z.B|.._.....
.2..i..hHTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.13.9
Date: Thu, 24 Sep 2026 12:23:15 GMT
Content-type: text/html
Content-Length: 38
Last-Modified: Thu, 24 Sep 2026 12:21:07 GMT

08:23:15.832793 IP 127.0.0.1.39222 > 127.0.0.1.8080: Flags [.], ack 186, win 256, options [nop,nop,TS val 1776672683 ecr 2603746180], length 0
E..4..@..@.....6.....Z.C5.....(.....
i...2..
08:23:15.832987 IP 127.0.0.1.8080 > 127.0.0.1.39222: Flags [P.], seq 186:224, ack 79, win 256, options [nop,nop,TS val 2603746180 ecr 1776672683], length 38: HTTP
E..Z^..@..@.....6Z.C5.....N.....
.2..i...<h1>CORE102 authorized test page</h1>

08:23:15.832996 IP 127.0.0.1.39222 > 127.0.0.1.8080: Flags [.], ack 224, win 256, options [nop,nop,TS val 1776672683 ecr 2603746180], length 0
E..4..@..@.....6.....Z.C[.....(.....
i...2..
08:23:15.833110 IP 127.0.0.1.8080 > 127.0.0.1.39222: Flags [F.], seq 224, ack 79, win 256, options [nop,nop,TS val 2603746181 ecr 1776672683], length 0
E..4^..@..@.....6Z.C[.....(.....
.2..i...
08:23:15.833556 IP 127.0.0.1.39222 > 127.0.0.1.8080: Flags [F.], seq 79, ack 225, win 256, options [nop,nop,TS val 1776672684 ecr 2603746181], length 0
E..4..@..@.....6.....Z.C\.....(.....
i...2..
08:23:15.833588 IP 127.0.0.1.8080 > 127.0.0.1.39222: Flags [.], ack 80, win 256, options [nop,nop,TS val 2603746181 ecr 1776672684], length 0
E..4^..@..@..!.....6Z.C\.....(.....
.2..i...

(kali@kali)-[~/core102_lab3]
$

```

```

# Terminal 1
cd ~/core102_lab3
echo '<h1>CORE102 authorised test page</h1>' > index.html
python3 -m http.server 8080 --bind 127.0.0.1

# Terminal 2
cd ~/core102_lab3
sudo tcpdump -ni lo -s0 -w core102_http.pcap 'tcp port 8080'

# Terminal 3
curl -v http://127.0.0.1:8080/

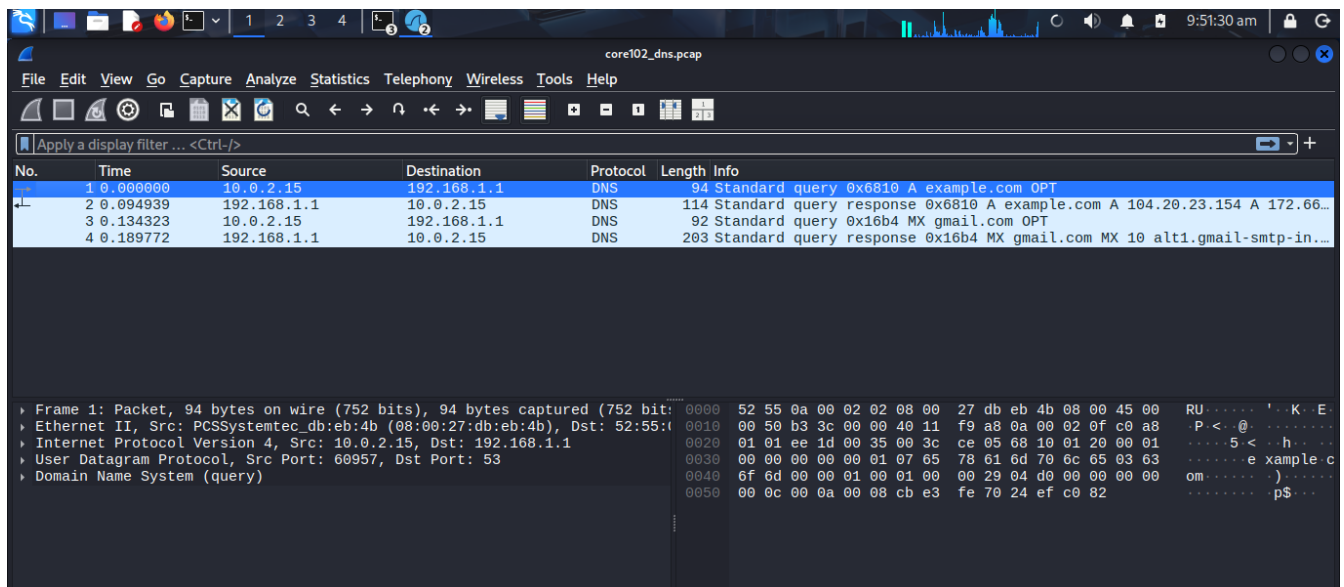
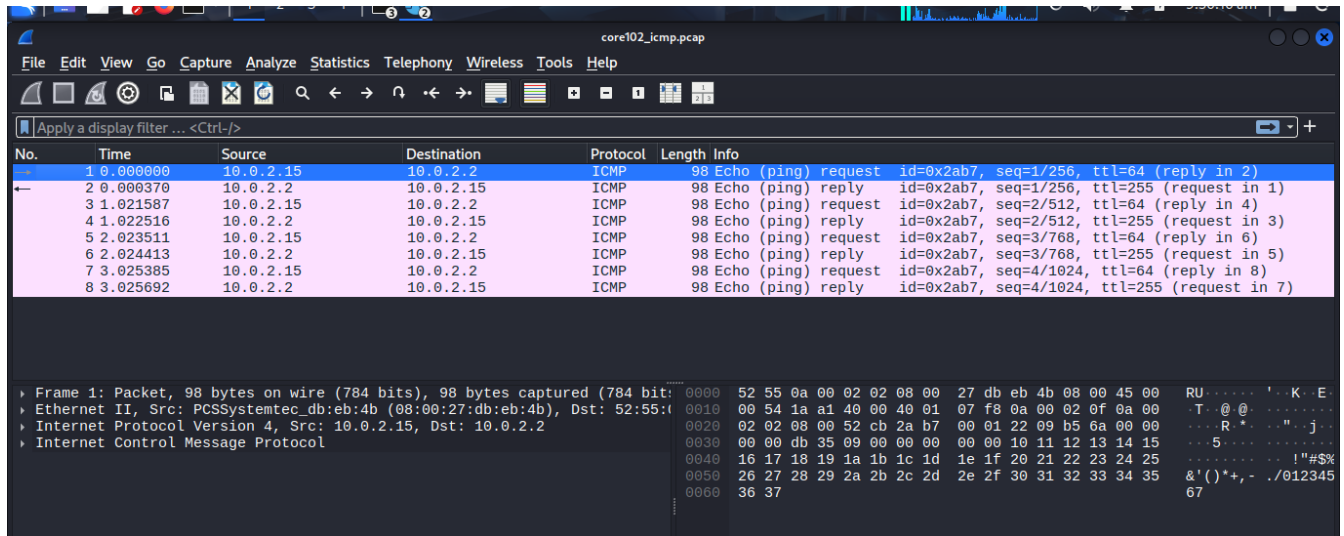
# After stopping capture with Ctrl+C
sudo tcpdump -nn -A -r core102_http.pcap 'tcp port 8080'

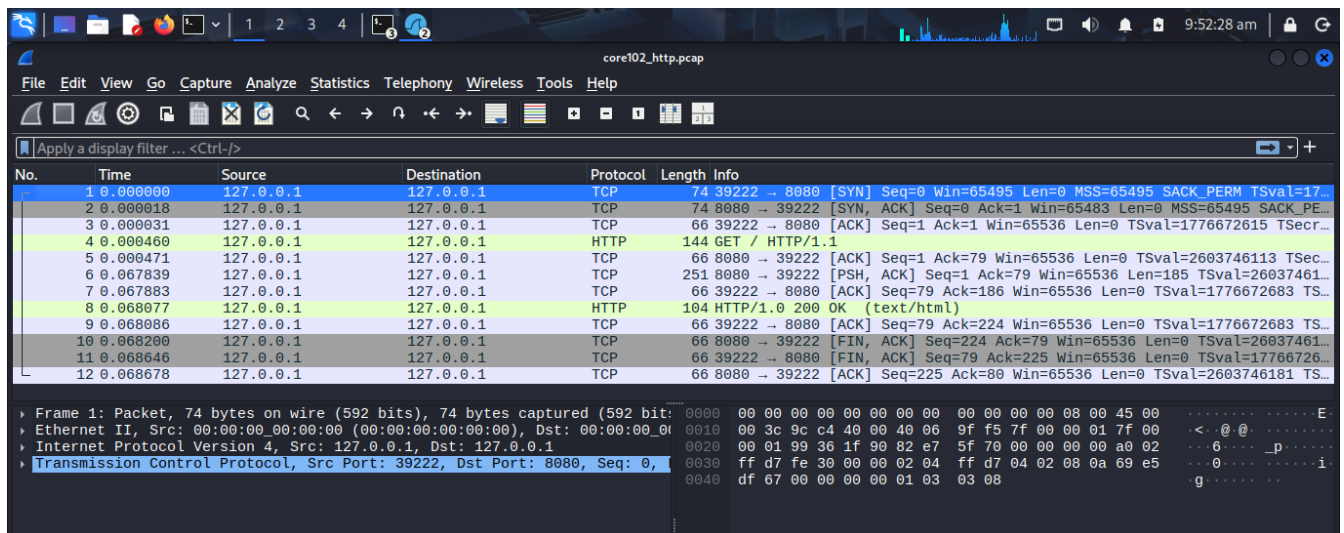
```

Stage	Display filter	Frame number	Evidence observed
Initial SYN	tcp.flags.syn == 1 && tcp.flags.ack == 0	1	Flags [S] from 127.0.0.1:ephemeral → 127.0.0.1:8080
SYN ACK	tcp.flags.syn == 1 && tcp.flags.ack == 1	2	Flags [S.] from server to client
Final ACK	tcp.flags.ack == 1	3	Flags [.] completes handshake
HTTP request	http.request	4	GET / HTTP/1.1 with Host: header
HTTP response	http.response	5	HTTP/1.0 200 OK + body

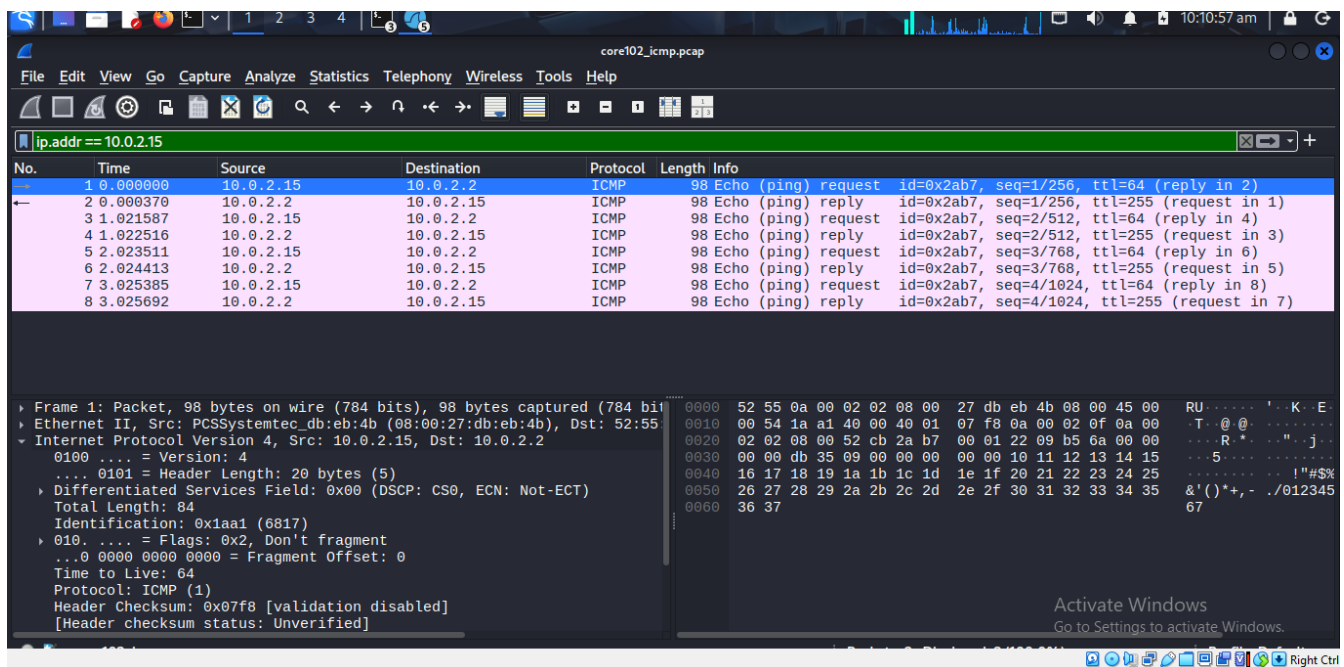
Part 5 Wireshark Analysis

1. Open each PCAP in Wireshark and confirm the file name before analysis.





2. Apply the filters below one at a time and capture a screenshot showing the filter bar, packet list and protocol details.



core102_icmp.pcap

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

icmp

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.2.15	10.0.2.2	ICMP	98	Echo (ping) request id=0x2ab7, seq=1/256, ttl=64 (reply in 2)
2	0.000370	10.0.2.2	10.0.2.15	ICMP	98	Echo (ping) reply id=0x2ab7, seq=1/256, ttl=255 (request in 1)
3	1.021587	10.0.2.15	10.0.2.2	ICMP	98	Echo (ping) request id=0x2ab7, seq=2/512, ttl=64 (reply in 4)
4	1.022516	10.0.2.2	10.0.2.15	ICMP	98	Echo (ping) reply id=0x2ab7, seq=2/512, ttl=255 (request in 3)
5	2.023511	10.0.2.15	10.0.2.2	ICMP	98	Echo (ping) request id=0x2ab7, seq=3/768, ttl=64 (reply in 6)
6	2.024413	10.0.2.2	10.0.2.15	ICMP	98	Echo (ping) reply id=0x2ab7, seq=3/768, ttl=255 (request in 5)
7	3.025385	10.0.2.15	10.0.2.2	ICMP	98	Echo (ping) request id=0x2ab7, seq=4/1024, ttl=64 (reply in 8)
8	3.025692	10.0.2.2	10.0.2.15	ICMP	98	Echo (ping) reply id=0x2ab7, seq=4/1024, ttl=255 (request in 7)

Ethernet II, Src: PCSSystemtec_db:eb:4b (08:00:27:db:eb:4b), Dst: 52:55:0000 52 55 0a 00 02 02 08 00 27 db eb 4b 08 00 45 00 RU.....K..E..

Internet Protocol Version 4, Src: 10.0.2.15, Dst: 10.0.2.2 0010 00 54 1a a1 40 00 40 01 07 f8 0a 00 02 0f 0a 00 .T.@.@.....

Internet Control Message Protocol 0020 02 02 08 00 52 cb 2a b7 00 01 22 09 b5 6a 00 00 ...R*.....j..

Type: Echo (ping) request (8) 0030 00 00 db 35 09 00 00 00 00 00 10 11 12 13 14 15 ...5.....!"#\$%&()*+,-./01234567

Code: 0

Checksum: 0x52cb [correct]

Checksum Status: Good

Identifier (BE): 10935 (0x2ab7)

Identifier (LE): 46890 (0xb72a)

Sequence Number (BE): 1 (0x0001)

Sequence Number (LE): 256 (0x0100)

[Response frame: 2]

ICMP Data: 2209b56a00000000db350900000000101112131415161718191a1b1c

Activate Windows
Go to Settings to activate Windows.

core102_http.pcap

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tcp.port == 8080

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	127.0.0.1	127.0.0.1	TCP	74	39222 → 8080 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=17...
2	0.000018	127.0.0.1	127.0.0.1	TCP	74	8080 → 39222 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PER...
3	0.000031	127.0.0.1	127.0.0.1	TCP	66	39222 → 8080 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=1776672615 TSecr...
4	0.000460	127.0.0.1	127.0.0.1	HTTP	144	GET / HTTP/1.1
5	0.000471	127.0.0.1	127.0.0.1	TCP	66	8080 → 39222 [ACK] Seq=1 Ack=79 Win=65536 Len=0 TSval=2603746113 TSec...
6	0.067839	127.0.0.1	127.0.0.1	TCP	251	8080 → 39222 [PSH, ACK] Seq=1 Ack=79 Win=65536 Len=185 TSval=26037461...
7	0.067883	127.0.0.1	127.0.0.1	TCP	66	39222 → 8080 [ACK] Seq=79 Ack=186 Win=65536 Len=0 TSval=1776672683 TS...
8	0.068077	127.0.0.1	127.0.0.1	HTTP	104	HTTP/1.0 200 OK (text/html)
9	0.068086	127.0.0.1	127.0.0.1	TCP	66	39222 → 8080 [ACK] Seq=79 Ack=224 Win=65536 Len=0 TSval=1776672683 TS...
10	0.068200	127.0.0.1	127.0.0.1	TCP	66	8080 → 39222 [FIN, ACK] Seq=224 Ack=79 Win=65536 Len=0 TSval=26037461...
11	0.068646	127.0.0.1	127.0.0.1	TCP	66	39222 → 8080 [FIN, ACK] Seq=79 Ack=225 Win=65536 Len=0 TSval=17766726...
12	0.068678	127.0.0.1	127.0.0.1	TCP	66	8080 → 39222 [ACK] Seq=225 Ack=80 Win=65536 Len=0 TSval=2603746181 TS...

Frame 4: Packet, 144 bytes on wire (1152 bits), 144 bytes captured (1152 bits) on interface 0

Ethernet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: 00:00:00:00:00:00

Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

Transmission Control Protocol, Src Port: 39222, Dst Port: 8080, Seq: 1, Len: 144

Source Port: 39222

Destination Port: 8080

[Stream index: 0]

[Stream Packet Number: 4]

[Conversation completeness: Complete, WITH_DATA (31)]

[TCP Segment Len: 78]

Sequence Number: 1 (relative sequence number)

Sequence Number (raw): 2196201329

[Next Sequence Number: 79 (relative sequence number)]

Acknowledgment Number: 1 (relative ack number)

0000 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00E..

0010 00 82 9c c6 40 00 40 06 9f ad 7f 00 00 01 7f 00 ...@.@.....

0020 00 01 99 36 1f 90 82 e7 5f 71 5a c9 42 7c 80 18 ...6.....qZ B|...

0030 01 00 fe 76 00 00 01 01 08 0a 69 e5 df 68 9b 32 ...v.....i..h 2

0040 03 40 47 45 54 20 2f 20 48 54 54 50 2f 31 2e 31 ...@GET / HTTP/1.1

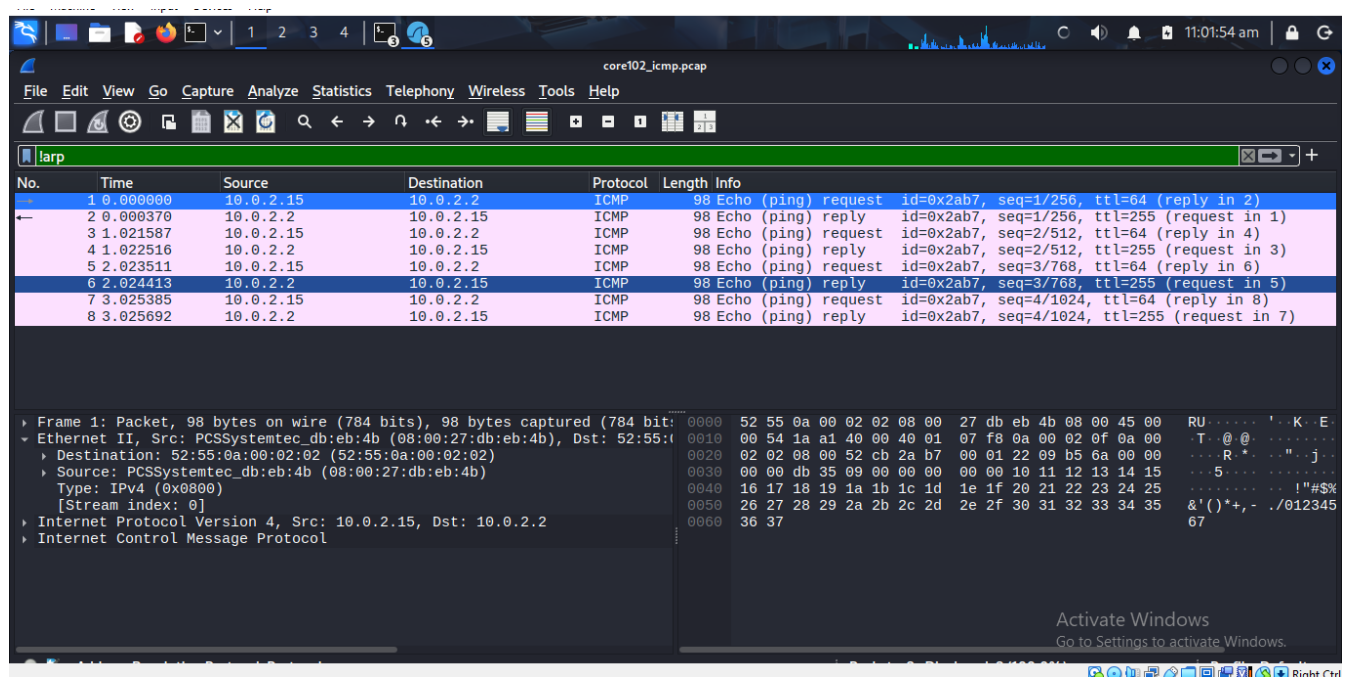
0050 0d 0a 48 6f 73 74 3a 20 31 32 37 2e 30 2e 30 2e ...Host: 127.0.0.

0060 31 3a 38 30 38 30 0d 0a 55 73 65 72 2d 41 67 65 ...1:8080 User-Age

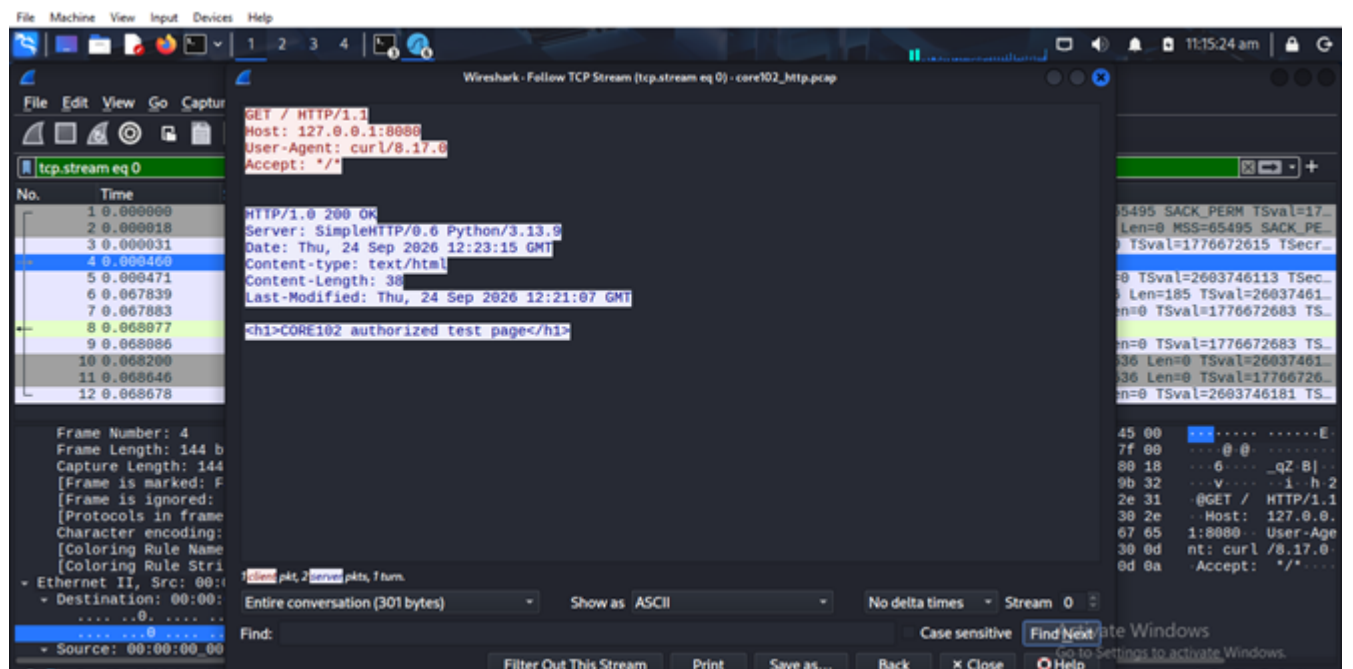
0070 6e 74 3a 20 63 75 72 6c 2f 38 2e 31 37 2e 30 0d ...nt: curl /8.17.0

0080 0a 41 63 63 65 70 74 3a 20 2a 2f 2a 0d 0a 0d 0a ...Accept: /*.....

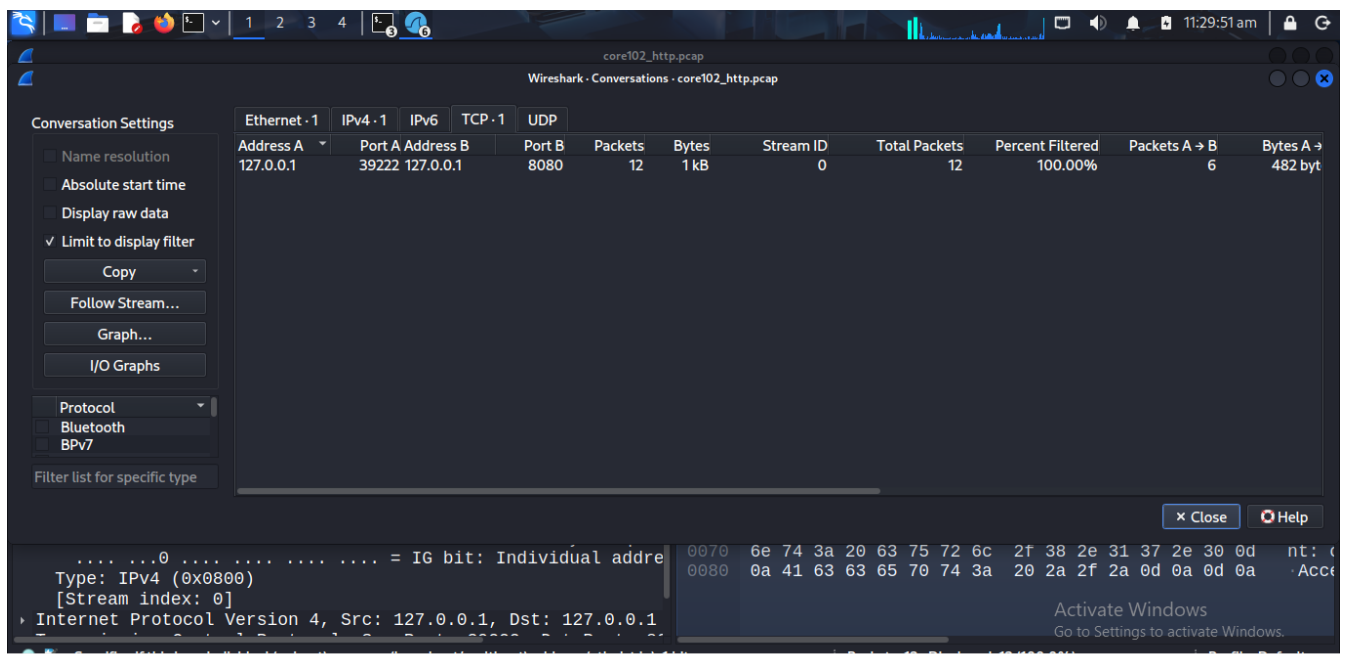
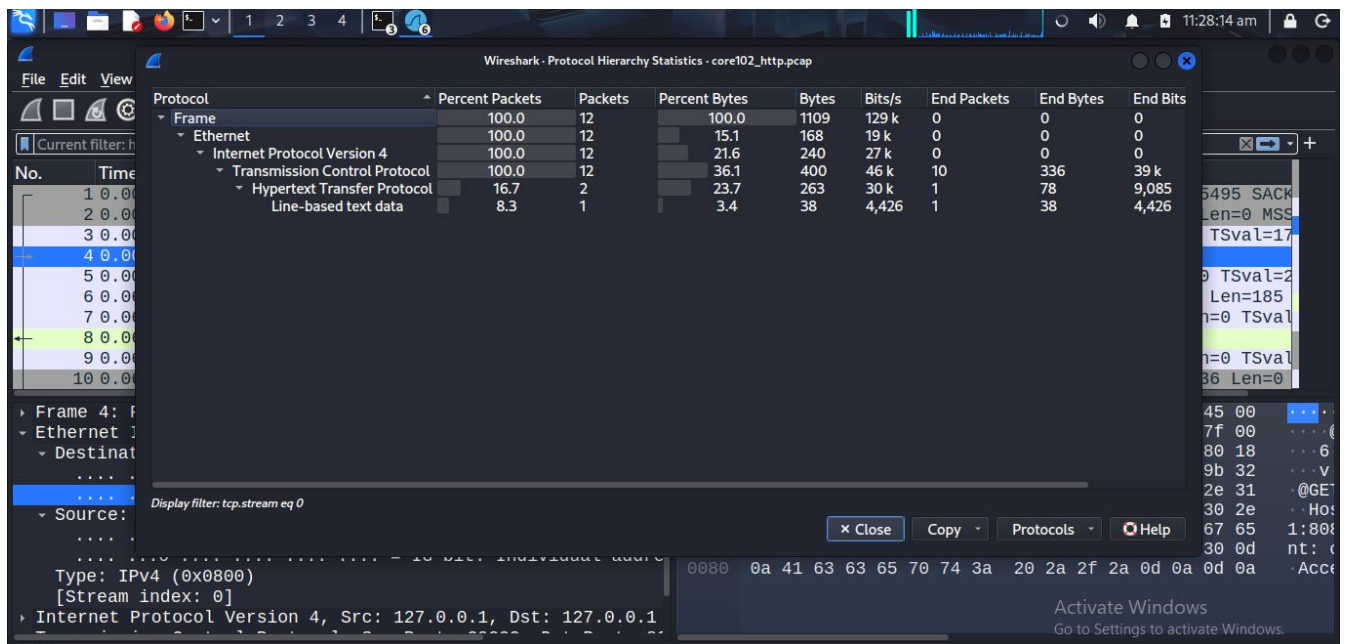
Activate Windows
Go to Settings to activate Windows.



- For one TCP conversation, use Follow TCP Stream and confirm that only your authorised test content appears.



- Use Statistics, Protocol Hierarchy and Conversations to summarise the capture.



Purpose	Wireshark display filter
Traffic to or from a host	ip.addr == 192.168.56.1
DNS traffic	dns
ICMP traffic	icmp
Local HTTP port	tcp.port == 8080

Purpose	Wireshark display filter
Initial TCP SYN packets	<code>tcp.flags.syn == 1 && tcp.flags.ack == 0</code>
TCP retransmissions	<code>tcp.analysis.retransmission</code>
Exclude ARP	<code>!arp</code>

Explain the difference between a capture filter and a display filter. Include one example of each and state when it is applied.

Answer

A **capture filter** uses BPF syntax and is applied **before** capture — packets that don't match are dropped at the interface and never reach the PCAP. **Example:** `tcp port 8080` (used with `tcpdump` in Part 4). It is applied at capture time and cannot be reversed.

A **display filter** uses Wireshark syntax and is applied **after** capture — packets remain in the file, only the view is filtered. **Example:** `tcp.port == 8080` (used in Wireshark in Part 5). It is applied post-capture and can be changed at any time.

A capture filter is applied before packets are captured, using BPF (Berkeley Packet Filter) syntax. Any packet that does not match is discarded at the network interface and never written to the PCAP file. It is applied at capture time and cannot be reversed.
Example: `tcp port 8080` — used with `tcpdump` in Part 4
(`sudo tcpdump -ni lo -s0 -w core102_http.pcap 'tcp port 8080'`).

A display filter is applied after capture, using Wireshark's own filter syntax. It only hides packets from view — the PCAP file remains unchanged and filters can be modified or removed at any time.
Example: `tcp.port == 8080` — used in Wireshark in Part 5 to view only the HTTP traffic inside `core102_http.pcap`.

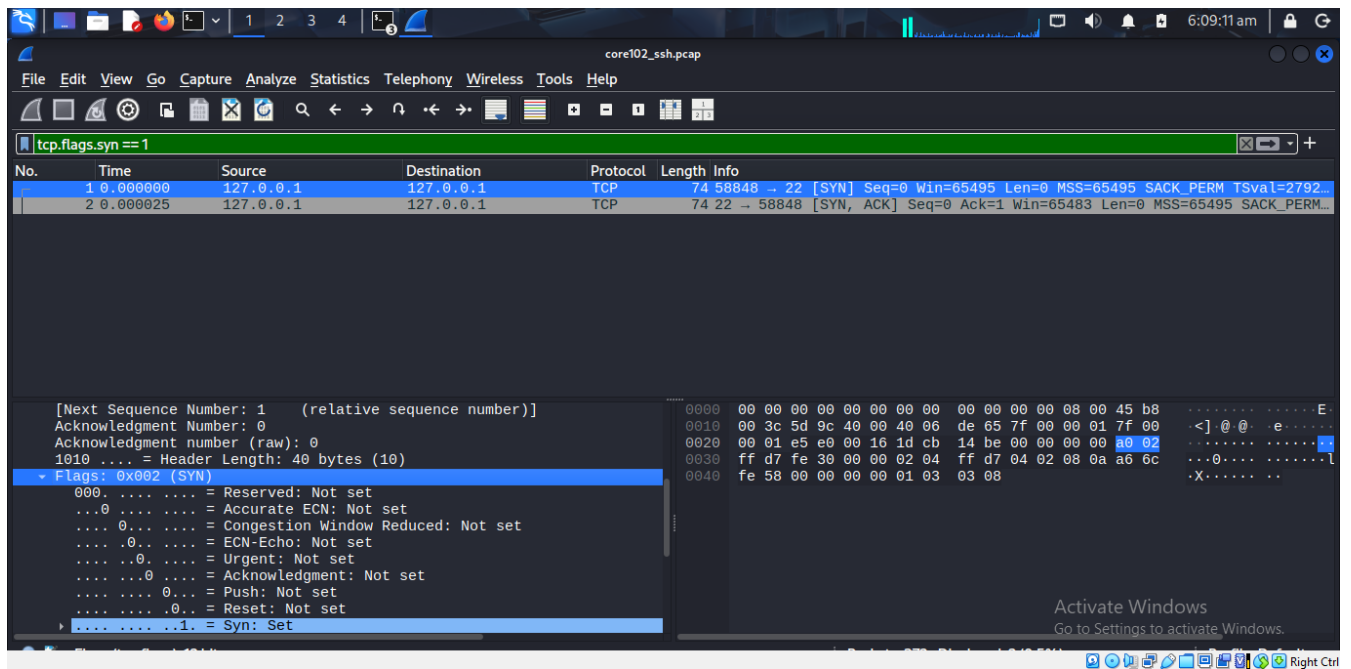
Part 6 Authorised SSH Observation

Scope: Use only your own second virtual machine or an instructor-provided capture.

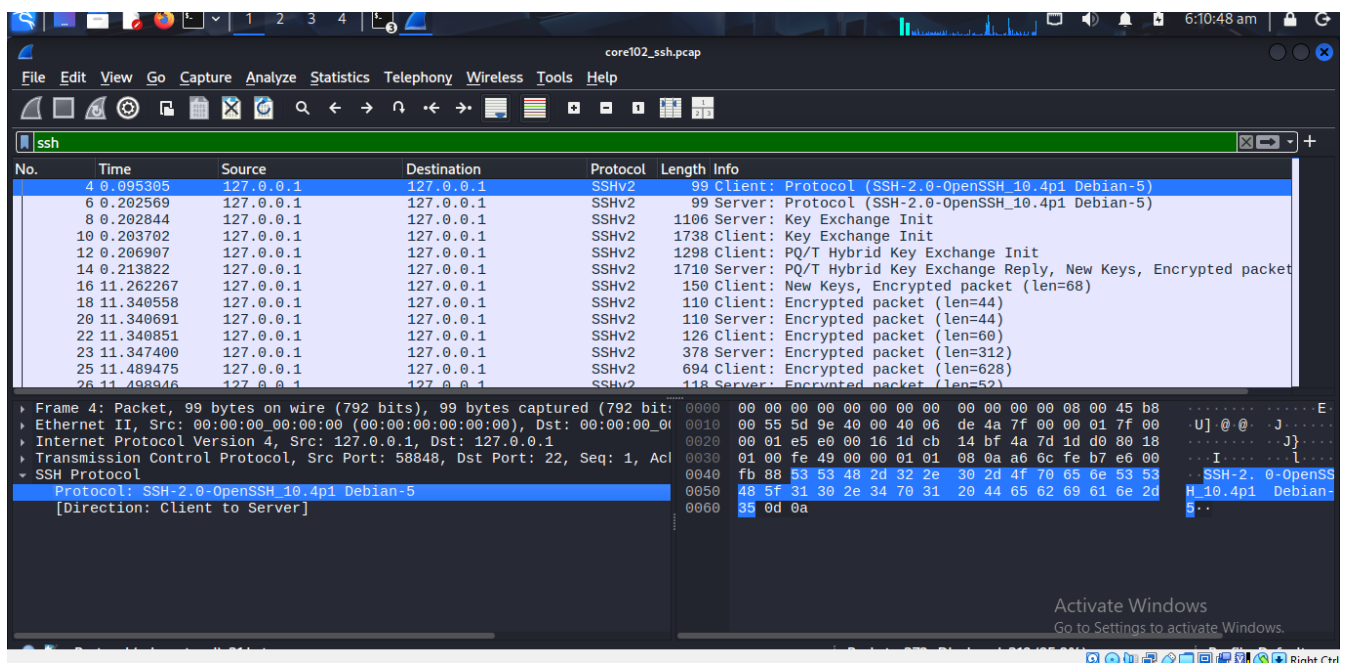
```
(kali@kali)-[~/core102_lab3]
$ sudo tcpdump -nn -r core102_ssh.pcap 'tcp port 22' | wc -l
reading from file core102_ssh.pcap, link-type EN10MB (Ethernet), snapshot length 262144
372

(kali@kali)-[~/core102_lab3]
$
```

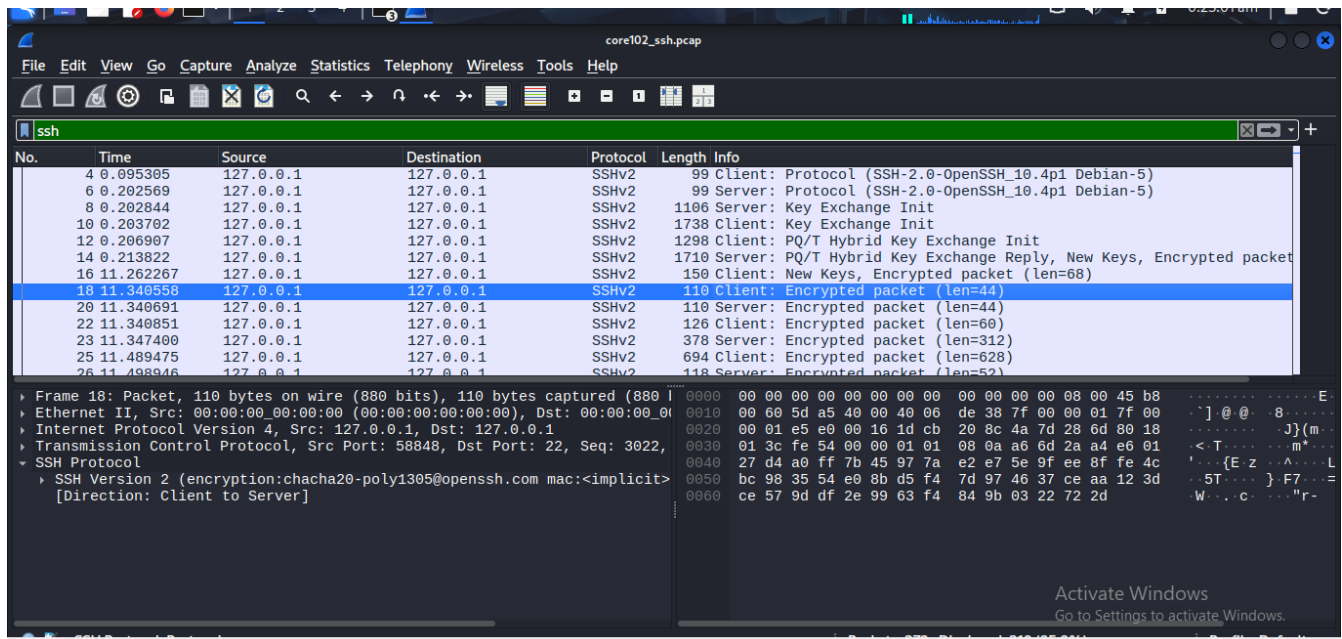
1. Start a `tcpdump` capture limited to TCP port 22 on the private-lab interface.



2. Connect to the second VM with SSH, run hostname and whoami, then exit.



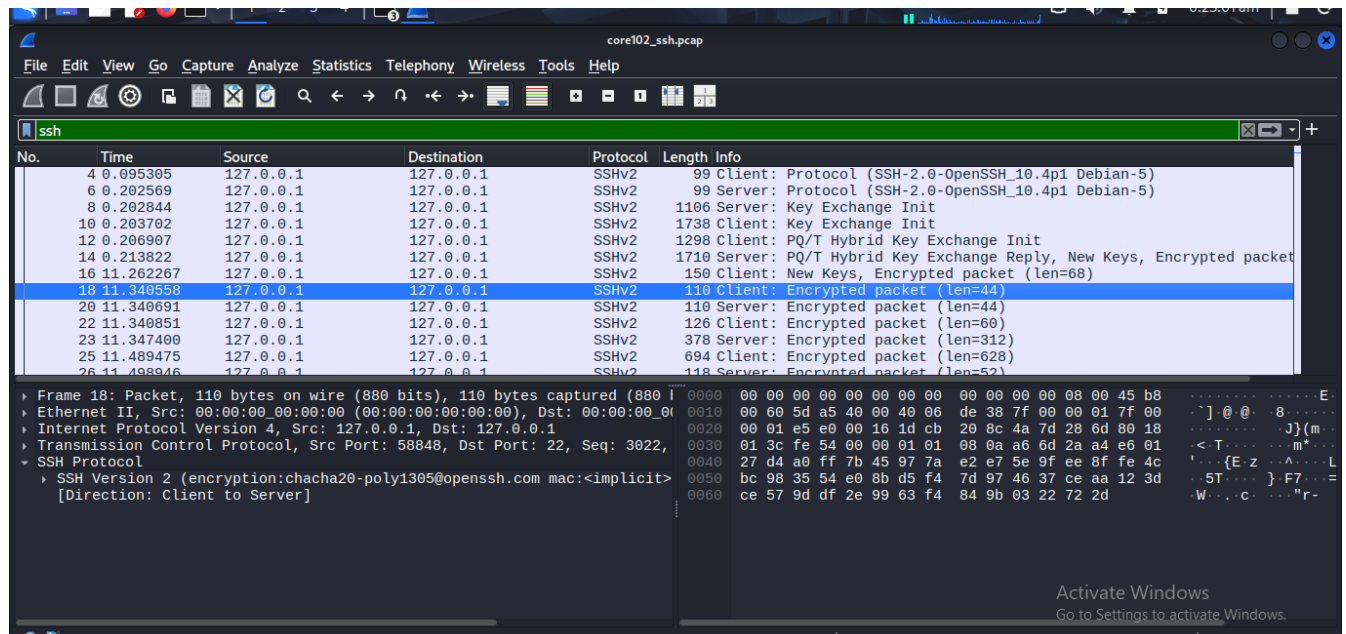
3. Stop the capture and identify the TCP handshake, encrypted payload packets and closure.



4. Explain why the commands and password are not readable in the packet payload.

ANSWER

SSH v2 encrypts everything after the initial version banner. Wireshark can see the TCP headers (ports, sequence numbers, flags) and the SSH version string (SSH-2.0-OpenSSH_10.4p1 Debian-5), but the username, password, commands, and their output all travel inside **encrypted SSH channel data** — Wireshark labels them simply as SSHv2 ... Encrypted packet (len=36) with no readable content.



```
sudo tcpdump -ni PRIVATE_IFACE -s0 -w core102_ssh.pcap 'tcp port 22'
# Second terminal
ssh student@192.168.56.1
hostname
whoami
exit
```

Part 7 Traffic Timeline Challenge

Use at least one of your PCAP files to reconstruct a short timeline. Reference exact frame numbers and timestamps.

Time	Frame	Source	Destination	Protocol	Event and interpretation
0.000000	1	127.0.0.1:39222	127.0.0.1:8080	TCP	SYN — client initiates TCP handshake with the HTTP server
0.000018	2	127.0.0.1:8080	127.0.0.1:39222	TCP	SYN-ACK — server accepts, sends its own sequence number
0.000031	3	127.0.0.1:39222	127.0.0.1:8080	TCP	ACK — handshake complete, connection ESTABLISHED
0.000360	4	127.0.0.1:39222	127.0.0.1:8080	HTTP	GET / HTTP/1.1 — client sends the HTTP request
0.000471	5	127.0.0.1:8080	127.0.0.1:39222	TCP	ACK — server acknowledges the request
0.067839	6	127.0.0.1:8080	127.0.0.1:39222	TCP	PSH-ACK — server begins pushing the response

1. Identify the first communication attempt.

Answer:

The first communication attempt is **frame 1** (time 0.000000), the initial **SYN** from 127.0.0.1:39222 (client) to 127.0.0.1:8080 (server). This is the first packet that begins the TCP three-way handshake for the HTTP session in core102_http.pcap.

2. Show the request and its matching response.

Answer

- **Request: frame 4** — GET / HTTP/1.1 with Host: 127.0.0.1:8080 and User-Agent: curl/8.17.0.
- **Response: frame 8** — HTTP/1.0 200 OK (text/html) with body <h1>CORE102 authorized test page</h1>.

3. Identify one field that links the packets into a conversation.

Answer

The **TCP source/destination port pair** links the packets: client port 39222 and server port 8080. Every packet in this capture belongs to the same TCP stream, identified by Wireshark as **Stream index 0**. The **TCP sequence and acknowledgment numbers** provide a second link — each request's sequence is acknowledged by the matching response's ack field.

4. State whether the exchange completed normally and cite the packet evidence.

Answer

Yes — the exchange completed normally.

- The **handshake** completed at frame 3 (client's final ACK).
- The **HTTP request** was sent at frame 4 and acknowledged at frame 5.
- The **HTTP response** was delivered at frame 8 with **200 OK** and the correct body.
- The **connection closed gracefully** at frames 10–12 with **FIN-ACK** in both directions followed by a final ACK. No **RST** (reset) appears, confirming a clean shutdown.

5. Calculate the elapsed time between the first request and final response.

Answer

The first request is frame 1 at time 0.000000 s. The final close (last ACK, frame 12) is at 0.068678 s.

Elapsed time = 0.068678 s $\approx$ 68.7 ms — the complete HTTP request-response cycle over loopback completed in under 70 milliseconds.

Troubleshooting Guide

Problem	Likely cause	Check or correction
tcpdump shows no packets	Wrong interface or overly narrow filter	Use tcpdump -D and test without -w using a small packet count
Permission denied	Capture requires elevated privilege	Use sudo only for the capture command
PCAP is empty	Traffic generated before capture started	Start capture first and verify listening message
Wireshark shows names instead of IPs	Name resolution enabled	Disable network name resolution for consistent evidence
HTTP filter shows nothing	Traffic is on loopback port 8080	Open core102_http.pcap and apply tcp.port == 8080

Assessment and Submission

Marking Guide

Assessment area	Evidence expected	Marks
Capture plan and baseline	Correct interfaces, filters, expected traffic and stop conditions	10
ICMP capture	Complete PCAP and accurate request/reply analysis	15
DNS capture	Correct query/response evidence and DNS interpretation	15
HTTP and TCP analysis	Controlled capture, handshake identification and HTTP evidence	20
Wireshark analysis	Effective display filters, statistics and protocol-field explanations	20
SSH or supplied capture analysis	Accurate encrypted-session and TCP-state observations	10
Timeline and report	Frame-supported timeline, clear evidence and complete reflection	10

Total: 100 marks

Submission Package

Submit one PDF report and one ZIP archive. Use your registration number in both filenames.

- REGNO_CORE102_Lab3_Report.pdf
- REGNO_CORE102_Lab3_Files.zip containing all authorised PCAP files and index.html

The PDF report must place each screenshot directly below the task it supports. Every screenshot must show the command and its output. Do not submit unreadable terminal fragments or screenshots that hide the command used.

Reflection Questions

1. How does a capture filter differ from a Wireshark display filter?

Answer

A **capture filter** is applied *before* packets are captured, using BPF (Berkeley Packet Filter) syntax. Any packet that does not match is discarded at the network interface and never written to the PCAP file. Example: `tcp port 8080` used with `tcpdump` in Part 4. Capture filters reduce data volume at the source and cannot be changed once the capture has run.

A **display filter** is applied *after* capture, using Wireshark's own filter syntax. It only hides packets from view — the PCAP file remains unchanged and filters can be modified or removed at any time. Example: `tcp.port == 8080` applied in Part 5. Display filters are used to explore and analyse data that has already been captured.

2. Which fields allowed you to pair a request with its response?

Answer

Different protocols use different linking fields:

- **ICMP:** the **identifier** and **sequence number** match between echo request and echo reply.
- **DNS:** the **Transaction ID** in the DNS header is identical in query and response, and the response is returned to the same client source port.

- **TCP/HTTP:** the **TCP 4-tuple** (source IP, source port, destination IP, destination port) plus the **sequence and acknowledgment numbers** link packets into a single TCP stream. Wireshark identifies this as the "Stream index."

3. What privacy risk exists when packet captures include unrelated traffic?

Answer

Unencrypted captures can expose **third-party personal data** — usernames, passwords, session cookies, authentication tokens, email content, visited URLs, and internal IP addresses. Sharing or uploading such PCAPs can breach privacy laws (GDPR, NDPR), the lab's ethics policy, and any applicable NDAs. To mitigate: use capture filters to limit scope, sanitize PCAPs before sharing, and never upload captures that include production or third-party traffic.

Final Checklist

- [x] I worked only in an authorised laboratory environment.
- [x] I included my full name and registration number in the report.
- [x] Every required command and result is visible in the evidence.
- [x] I explained results instead of submitting screenshots alone.
- [x] I removed passwords, keys, tokens and unrelated personal information.
- [x] I opened the submitted files and confirmed that they are complete.